

Namo

Named dimensional data for Ruby.

Namo is a Ruby library for working with multi-dimensional data using named dimensions. It infers dimensions and coordinates from plain arrays of hashes — the same shape you get from databases, CSV files, JSON, and YAML — so there's no reshaping step.

The design rests on a few stances: every hash key is a dimension and none is privileged as a coordinate or value; formulae attach to a Namu alongside data and re-evaluate on each access, appearing as derived dimensions alongside the data dimensions; operators that combine Namos all take Namos and return Namos, so analytical pipelines close; and the formula mechanism is type-agnostic — strings, dates, booleans, and arbitrary Ruby objects work as readily as numbers.

Installation

```
gem install namo
```

Or in your Gemfile:

```
gem 'namo'
```

Usage

Create a Namu instance from an array of hashes:

```
require 'namo'

sales = Namu.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])
```

Dimensions and coordinates are inferred:

```
sales.dimensions
# => [:product, :quarter, :price, :quantity]
```

```
sales.coordinates[:product]
# => ['Widget', 'Gadget']
```

```
sales.coordinates[:quarter]
# => ['Q1', 'Q2']
```

Every key is a dimension; every value is a coordinate. There's no schema declaration and no choosing which column is “the index” — price and quantity are no less first-class than product and quarter.

Selection

Select by named dimension using keyword arguments:

```
# Single value
sales[product: 'Widget']
# => #<Namu [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>
```

```
# ]>

# Multiple dimensions
sales[product: 'Widget', quarter: 'Q1']
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100}
# ]>

# Range
sales[price: 10.0..20.0]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>

# Array of values
sales[quarter: ['Q1']]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
# ]>
```

Projection

Project to specific dimensions:

```
sales[:,product, :price]
# => #<Namo [
#   {product: 'Widget', price: 10.0},
#   {product: 'Widget', price: 10.0},
#   {product: 'Gadget', price: 25.0},
#   {product: 'Gadget', price: 25.0}
# ]>
```

Selection and projection can be chained:

```
sales[product: 'Widget'][:,quarter, :price]
# => #<Namo [
#   {quarter: 'Q1', price: 10.0},
#   {quarter: 'Q2', price: 10.0}
# ]>
```

Or combined in a single call (names before selectors):

```
sales[:,quarter, :price, product: 'Widget']
# => #<Namo [
#   {quarter: 'Q1', price: 10.0},
#   {quarter: 'Q2', price: 10.0}
# ]>
```

Contraction

Contraction is the complement of projection. Projection says “keep these dimensions”; contraction says “remove these dimensions, keep everything else”:

```
sales[-:price, -:quantity]
# => #<Namo [
```

```
# {product: 'Widget', quarter: 'Q1'},
# {product: 'Widget', quarter: 'Q2'},
# {product: 'Gadget', quarter: 'Q1'},
# {product: 'Gadget', quarter: 'Q2'}
# ]>
```

The `-:price` syntax uses unary minus on `Symbol` to produce a negated dimension. Mixing projection and contraction in the same call is an error — the two modes are mutually exclusive:

```
sales[:product, -:price] # => ArgumentError
```

Selection and contraction can be chained:

```
sales[product: 'Widget'][-:price, -:quantity]
# => #<Namo [
# {product: 'Widget', quarter: 'Q1'},
# {product: 'Widget', quarter: 'Q2'}
# ]>
```

Or combined in a single call (names before selectors):

```
sales[-:price, -:quantity, product: 'Widget']
# => #<Namo [
# {product: 'Widget', quarter: 'Q1'},
# {product: 'Widget', quarter: 'Q2'}
# ]>
```

Selection, projection, and contraction always return a new `Namo` instance, so everything chains.

Concatenation

`+` is the first of `Namo`'s binary operators: it takes a `Namo` on each side and returns a `Namo`. The same shape holds for `-`, `&`, `|`, `^`, `==`, `===`, `<`, `<=`, `>`, `>=` and (later) the composition operators — `Namo` in, `Namo` (or boolean) out — so analytical pipelines stay queryable end-to-end.

`+` combines two `Namo` objects that share the same dimensions by appending the rows of the second to the first:

```
q1_sales = Namu.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
])
```

```
q2_sales = Namu.new([
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])
```

```
all_sales = q1_sales + q2_sales
# => #<Namo [
# {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
# {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
# {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
# {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
# ]>
```

The dimensions must match — concatenating `Namo` objects with different dimensions raises an `ArgumentError`. Formulae carry through from the left-hand side.

Row Removal

– removes from the first Namo any row that appears exactly in the second:

```
sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])

discontinued = Namo.new([
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])

sales - discontinued
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>
```

Removal is exact — every dimension, every value must match. The dimensions must match; different dimensions raise an `ArgumentError`. Formulae carry through from the left-hand side.

Intersection

& returns the rows present in both Namo objects, like `Array#&`:

```
sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])

confirmed = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])

sales & confirmed
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
# ]>
```

The dimensions must match; different dimensions raise an `ArgumentError`. Formulae carry through from the left-hand side.

Union

| returns all rows from both sides, deduplicated, like `Array#|`:

```
q1_sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
])
```

```

])

all_sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Thingo', quarter: 'Q3', price: 5.0, quantity: 10}
])

q1_sales | all_sales
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
#   {product: 'Thingo', quarter: 'Q3', price: 5.0, quantity: 10}
# ]>

```

The dimensions must match; different dimensions raise an `ArgumentError`. Formulae merge from both sides; the left-hand side's formulae take precedence on conflict.

Symmetric Difference

`^` returns rows that appear in one side but not both:

```

set_a = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
])

set_b = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Thingo', quarter: 'Q3', price: 5.0, quantity: 10}
])

set_a ^ set_b
# => #<Namo [
#   {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
#   {product: 'Thingo', quarter: 'Q3', price: 5.0, quantity: 10}
# ]>

```

The dimensions must match; different dimensions raise an `ArgumentError`. Formulae merge from both sides; the left-hand side's formulae take precedence on conflict.

Equality

Comparison on Namos is **multiset-theoretic on rows**: row order is ignored (it's an accident of ingestion, not data), but row multiplicities count (they *are* data). The same stance carries across the equality, pattern-match, and subset/superset operators below.

`==` is multiset equality on rows. Class and formulae are ignored; row order is ignored; row multiplicities are not.

```

a = Namo.new([{:x: 1}, {:x: 2}])
b = Namo.new([{:x: 2}, {:x: 1}])

a == b
# => true

a == Namo.new([{:x: 1}, {:x: 1}, {:x: 2}])
# => false

```

`eql?` is stricter: it also requires the class to match and the formula names to match. Like `===`, it ignores proc bodies — proc identity isn't a meaningful equivalence in Ruby (`proc{...} == proc{...}` is false), so neither `===` nor `eql?` uses it.

`hash` is consistent with `eql?` and is content-based, so equal Namos hash equally and can be used as Hash keys:

```
h = {a => 'first'}
h[b]
# => 'first'
```

`equal?` is unchanged from Ruby's default — it tests object identity.

`===` answers a different question: does the candidate have the same dimensions and the same formula names? Row data is ignored, and so are the proc bodies themselves — only the names matter. This is the `===` semantics that case statements use, so Namos can serve as templates for analytical shape:

```
sales_shape = Namo.new([{:product: 'X', quarter: 'Q1', price: 0.0, quantity: 0}])
sales_shape[:revenue] = proc{|row| row[:price] * row[:quantity]}
```

```
q1 = Namo.new([{:product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100}])
q1[:revenue] = proc{|row| row[:price] * row[:quantity]}
```

```
sales_shape === q1
# => true (same dimensions, same formula name)
```

```
sales_shape == q1
# => false (different rows)
```

The two `:revenue` procs are independently-written and not the same object — `proc{...} == proc{...}` is false in Ruby. But `===` doesn't compare proc identity; it asks “do these Namos have the same analytical shape?” and the shape is the set of dimensions plus the set of formula names.

Each comparison operator answers a distinct question: `eql?` is strictest (class + data + formula names); `==` is data identity; `===` is analytical identity; the subset operators are data containment.

Subset and Superset

`<`, `<=`, `>`, `>=` are multiset subset and superset relations on rows.

```
small = Namo.new([{:x: 1}, {:x: 2}])
large = Namo.new([{:x: 1}, {:x: 2}, {:x: 3}])
```

```
small <= large
# => true
```

```
small < large
# => true
```

```
large > small
# => true
```

Equal sets are `<=` and `>=` each other, but neither `<` nor `>`. Disjoint sets are none of the above — unless one side is empty, in which case it is a subset of (and disjoint with) the other.

Multiplicity matters: a single `{x: 1}` is a proper subset of two `{x: 1}`s.

```
one = Namo.new([{:x: 1}])
two = Namo.new([{:x: 1}, {:x: 1}])
```

```
one < two
# => true
```

The dimensions must match; different dimensions raise an `ArgumentError`. Comparing against a non-Namo raises a `TypeError`.

Formulae

Define computed dimensions using `[]=:`:

```
sales[:revenue] = proc{|row| row[:price] * row[:quantity]}

sales[:product, :quarter, :revenue]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', revenue: 1000.0},
#   {product: 'Widget', quarter: 'Q2', revenue: 1500.0},
#   {product: 'Gadget', quarter: 'Q1', revenue: 1000.0},
#   {product: 'Gadget', quarter: 'Q2', revenue: 1500.0}
# ]>
```

Formulae aren't materialised into row data — they re-evaluate on every access. A `:revenue` value reflects the current `:price` and `:quantity` at the moment you ask for it, so derived values stay in sync with whatever the underlying data is doing.

Formulae compose:

```
sales[:cost] = proc{|row| row[:quantity] * 4.0}
sales[:profit] = proc{|row| row[:revenue] - row[:cost]}

sales[:product, :quarter, :profit]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', profit: 600.0},
#   {product: 'Widget', quarter: 'Q2', profit: 900.0},
#   {product: 'Gadget', quarter: 'Q1', profit: 840.0},
#   {product: 'Gadget', quarter: 'Q2', profit: 1260.0}
# ]>
```

Formulae work with selection and projection:

```
sales[product: 'Widget'][:revenue, :quarter]
# => #<Namo [
#   {revenue: 1000.0, quarter: 'Q1'},
#   {revenue: 1500.0, quarter: 'Q2'}
# ]>
```

Formulae carry through selection — a filtered Namo instance remembers its formulae.

Coordinates and values

`dimensions` covers the *queryable namespace* — every name you can ask for, whether it lives in the row data or is computed by a formula. Once formulae are defined, they appear alongside data dimensions:

```
sales[:revenue] = proc{|row| row[:price] * row[:quantity]}

sales.dimensions
# => [:product, :quarter, :price, :quantity, :revenue]

sales.data_dimensions
```

```
# => [:product, :quarter, :price, :quantity]
```

```
sales.derived_dimensions
```

```
# => [:revenue]
```

coordinates gives the unique values per dimension, including derived ones:

```
sales.coordinates[:product]
```

```
# => ['Widget', 'Gadget']
```

```
sales.coordinates[:revenue]
```

```
# => [1000.0, 1500.0]
```

values gives the full per-row sequence — duplicates preserved, row order preserved:

```
sales.values[:product]
```

```
# => ['Widget', 'Widget', 'Gadget', 'Gadget']
```

```
sales.values[:revenue]
```

```
# => [1000.0, 1500.0, 1000.0, 1500.0]
```

Both coordinates and values accept positional arguments. With no args they return a Hash across the queryable namespace; with one arg they lazily compute and return just that column as an Array; with multiple args they return a subset Hash containing just the requested columns:

```
sales.values(:product)
```

```
# => ['Widget', 'Widget', 'Gadget', 'Gadget']
```

```
sales.values(:product, :quarter)
```

```
# => {
```

```
#   product: ['Widget', 'Widget', 'Gadget', 'Gadget'],
```

```
#   quarter: ['Q1', 'Q2', 'Q1', 'Q2']
```

```
# }
```

```
sales.coordinates(:revenue)
```

```
# => [1000.0, 1500.0]
```

Single-arg access is lazy: sales.values(:revenue) evaluates the formula only across the rows of :revenue, without materialising the other columns. The bracket form (sales.values[:revenue]) still works through ordinary Hash lookup but pays for the full materialisation up front.

coordinates is values with .uniq applied per column — coordinates(dim) == values(dim).uniq holds for every dimension.

to_h is the Ruby-conventional alias for the full values Hash:

```
sales.to_h
```

```
# => {
```

```
#   product: ['Widget', 'Widget', 'Gadget', 'Gadget'],
```

```
#   quarter: ['Q1', 'Q2', 'Q1', 'Q2'],
```

```
#   price: [10.0, 10.0, 25.0, 25.0],
```

```
#   quantity: [100, 150, 40, 60],
```

```
#   revenue: [1000.0, 1500.0, 1000.0, 1500.0]
```

```
# }
```

Unknown dimensions propagate nil per row — values(:missing) returns [nil, nil, ...] rather than raising or returning a sentinel, matching the convention used by Row#[] and [] selection. Use dimensions.include?(:dim) if you need to check membership directly.

Enumerable

Namo includes Enumerable, so each, reduce, map, select, min_by, and all the rest work out of the box. Rows are yielded as Row objects, so formulae are accessible during enumeration:

```
sales.reduce(0){|sum, row| sum + row[:quantity]}
# => 350

sales[product: 'Widget'].reduce(0){|sum, row| sum + row[:quantity]}
# => 250

sales[:revenue] = proc{|row| row[:price] * row[:quantity]}

sales.reduce(0){|sum, row| sum + row[:revenue]}
# => 5000.0

sales[product: 'Widget'].reduce(0){|sum, row| sum + row[:revenue]}
# => 2500.0

sales.map{|row| row[:product]}
# => ['Widget', 'Widget', 'Gadget', 'Gadget']

sales.min_by{|row| row[:price]}[:product]
# => 'Widget'

sales.flat_map{|row| [row[:price]]}
# => [10.0, 10.0, 25.0, 25.0]
```

Extracting data

to_a returns an array of hashes — the row-oriented form:

```
sales[:product, :quarter, :revenue].to_a
# => [
#   {product: 'Widget', quarter: 'Q1', revenue: 1000.0},
#   {product: 'Widget', quarter: 'Q2', revenue: 1500.0},
#   {product: 'Gadget', quarter: 'Q1', revenue: 1000.0},
#   {product: 'Gadget', quarter: 'Q2', revenue: 1500.0}
# ]
```

to_h returns a hash of arrays — the columnar form (see Coordinates and values above):

```
sales[:product, :quarter, :revenue].to_h
# => {
#   product: ['Widget', 'Widget', 'Gadget', 'Gadget'],
#   quarter: ['Q1', 'Q2', 'Q1', 'Q2'],
#   revenue: [1000.0, 1500.0, 1000.0, 1500.0]
# }
```

Why?

Every other multi-dimensional array library requires you to pre-shape your data before you can work with it. Namu takes it in the form it likely already comes in.

Name

Namo: nam(ed) (dimensi)o(ns). A companion to Numo (numeric arrays for Ruby). And in Aussie culture 'o' gets added to the end of names.

Contributing

1. Fork it (<https://github.com/thoran/namo/fork>)
2. Create your feature branch (git checkout -b my-new-feature)
3. Commit your changes (git commit -am 'Add some feature')
4. Push to the branch (git push origin my-new-feature)
5. Create a new pull request

License

MIT